Manual técnico de TechnoStore

Aprendices:

Danna Michelle Farfan Ramírez

Cristian Camilo Alegria Rivera

Kevin Camilo Caballero Oliveros

Deiner Smit Limas Lavao

Juan Manuel Silva Castañeda

Programa: Análisis y Desarrollo de Software (ADSO) - Ficha 3407799

Centro de Formación: Centro de la Industria, la Empresa y los Servicios (SENA)

Fecha: 09/2026

Tienda en línea con dinero digital simulado

Programa: Análisis y Desarrollo de Software (ADSO), ficha 3407799 **Centro de formación:** Centro de la Industria, la Empresa y los Servicios (SENA) **Aprendices:** Danna Michelle Farfan Ramírez, Cristian Camilo Alegria Rivera, Kevin Camilo Caballero Oliveros, Deiner Smit Limas Lavao, Juan Manuel Silva Castañeda **Versión del manual:** 1.2 · **Fecha:** noviembre de 2026 **Fuentes:** código del proyecto (Nueva_carpeta.zip), SRS v1.0 y README del repositorio

Control de versiones

Versión	Fecha	Descripción del cambio	Responsable
1.0	10/11/2026	Primera versión, derivada del SRS	Equipo de desarrollo
1.1	10/11/2026	Reestructuración técnica: arquitectura, instalación, contrato de la API y plan de pruebas	Equipo de desarrollo
1.2	10/11/2026	Verificación contra el código fuente. Corrección de la configuración, inventario real de acciones y de objetos de base de datos, estado real de las observaciones de seguridad y sección de hallazgos	Equipo de desarrollo

1. Objeto, alcance y fuentes

Este manual describe la arquitectura, la configuración, la instalación, el contrato de la API, el modelo de datos, la lógica transaccional, la seguridad y los procedimientos de prueba y mantenimiento de TechnoStore. Va dirigido a desarrolladores que mantienen el sistema, administradores que lo despliegan y evaluadores del proyecto formativo.

1.1 Fuentes y prioridad

La versión 1.2 se contrastó con el código del proyecto. Cuando el SRS y el código no coinciden, este manual sigue al código y registra la diferencia en la sección 15. Las referencias de línea (por ejemplo, `api.php:315`) corresponden a la versión del código revisada y pueden desplazarse si el archivo cambia.

1.2 Convenciones

- Los nombres de tablas, columnas, procedimientos, vistas, acciones y archivos aparecen en `código`.
- RF, RN, RNF y O remiten a los requerimientos funcionales, reglas de negocio, requerimientos no funcionales y observaciones técnicas del SRS.
- Los importes se expresan en pesos colombianos (COP). En la base de datos se guardan como DECIMAL(12,2). En la interfaz se muestran con punto como separador de miles, por ejemplo $1.599.000.

2. Arquitectura del sistema

TechnoStore es una aplicación web de tres capas. El navegador carga una sola página (`index.php`) y consume la API (`api.php`) mediante peticiones HTTP. La API resuelve la lógica de negocio y accede a MySQL o MariaDB mediante PDO.

2.1 Componentes y archivos

Capa	Archivo	Responsabilidad
Presentación	`index.php`	Interfaz de una sola página: catálogo, carrito, pedidos, perfil y panel administrativo
Lógica de aplicación	`api.php`	48 acciones identificadas por `action`, control de acceso, transacciones y migración de esquema
Configuración	`config.php`	Lectura de variables de entorno, conexión PDO y constantes de correo
Notificaciones	`mailer.php`	Correos de pedidos mediante PHPMailer y SMTP
Reportes	`reporte.php`	Reportes de pedidos, usuarios y productos
Excel	`xlsx.php`	Clase `XlsxWriter` que genera el libro .xlsx sin librerías externas
Datos	`bdcatalogo.sql`	Esquema inicial, vistas, procedimientos almacenados y datos de ejemplo
Dependencias	`composer.json`, `composer.lock`	Declaran `phpmailer/phpmailer` en versión `^7.1`
Documentación	`README.md`, `Anexos/`	Guía de instalación, diagrama MER (`Anexos/MER/`) y SRS (`Anexos/SRS/`)
Recursos	`banner/`, `logo/`	Imágenes institucionales de la tienda

Servicios externos: restcountries.com (lista de países) y Nominatim de OpenStreetMap (autocompletado de ciudades). Ambos tienen un tiempo máximo de espera de 12 segundos (`api.php:58`). Si fallan, la API devuelve listas vacías y el usuario completa país y ciudad a mano (RNF-09).

2.2 Flujo de una petición

1. El navegador envía una petición HTTP a `api.php` con el parámetro `action`.
2. La API verifica la sesión con `needLogin()` o `needAdmin()`, según la acción, y recorta las entradas de texto con `trim()`.
3. La acción ejecuta una consulta preparada, un procedimiento almacenado o una transacción.
4. La API responde JSON con la forma `{success: boolean, ...}`.
5. Si un pedido cambia de estado, `mailer.php` envía el correo después del COMMIT. Un fallo de correo no revierte el cambio.

2.3 Decisiones de diseño

- **Autenticación con sesiones de PHP:** el estado de autenticación vive en el servidor. Cerrar sesión con `session_destroy()` invalida el acceso de inmediato. No se usan tokens JWT.
- **Lógica crítica en la base de datos:** las reglas de stock y saldo se ejecutan en transacciones y procedimientos almacenados, cerca de los datos.
- **Bloqueo pesimista:** las filas de pedido, producto y usuario se bloquean con `SELECT ... FOR UPDATE` para evitar condiciones de carrera (RNF-02).
- **Baja lógica:** productos, categorías y usuarios se desactivan con una bandera. Así se conservan el historial, los comprobantes y los reportes (RN-14).
- **Precio congelado:** al confirmar, el precio se copia en `detalle_pedido.precio_unitario`. Los cambios posteriores no alteran pedidos existentes (RN-02).

3. Requisitos del entorno

Componente	Requisito	Observaciones
PHP	Versión por definir: 7.4 según el README, 8.0 según el SRS (RNF-12)	Ver hallazgo H-02 en la sección 15. Extensiones PDO MySQL, ZIP y de procesamiento de imágenes (GD o equivalente, necesaria para `getimagesize()`)
Base de datos	MySQL 8.0.16 o superior, o MariaDB 10.5 o superior	MySQL 8.0.16 es el mínimo para las restricciones CHECK. Motor InnoDB y collation `utf8mb4_unicode_ci`
Composer	Vigente	Instala `phpmailer/phpmailer` en versión `^7.1`
Servidor local de referencia	XAMPP con Apache	Entorno de desarrollo del proyecto
Navegadores	Últimas dos versiones de Chrome, Edge, Firefox y Safari	RNF-12
Zona horaria	`America/Bogotá`	La usan los reportes Excel

3.1 Verificación del entorno

```
php -v
php -m | grep -E "pdo_mysql|zip|gd"
mysql --version
```

En Windows con PowerShell, reemplace `grep -E` por `findstr /R`. La salida de `php -m` debe incluir `pdo_mysql`, `zip` y `gd`. Si falta alguno, habilítelo en `php.ini` y reinicie Apache.

4. Configuración

`config.php` no contiene credenciales. Lee cada parámetro de una variable de entorno con la función `configEnv()` y usa un valor por defecto si la variable no existe. Las variables se definen en el servidor web, por ejemplo con la directiva `SetEnv` de Apache, y nunca en el código fuente.

4.1 Variables de entorno

Variable	Valor por defecto en el código	Uso
`DB_HOST`	`127.0.0.1:8080`	Host del servidor MySQL. Admite el formato `host:puerto`
`DB_PORT`	`3306`	Puerto, usado cuando `DB_HOST` no incluye puerto
`DB_NAME`	`catalogo_ecommerce`	Nombre de la base de datos
`DB_USER`	`root`	Usuario de la base de datos
`DB_PASS`	vacío	Contraseña del usuario
`MAIL_ENABLED`	`false`	Activa el envío de correos. Acepta valores booleanos como `true` o `false`
`MAIL_HOST`	`smtp.gmail.com`	Servidor SMTP
`MAIL_PORT`	`587`	Puerto SMTP
`MAIL_USER`	vacío	Usuario SMTP
`MAIL_PASS`	vacío	Contraseña SMTP
`MAIL_FROM`	Igual a `MAIL_USER`	Dirección del remitente
`MAIL_FROM_NAME`	`TechnoStore`	Nombre del remitente

Atención: el valor por defecto de `DB_HOST` es `127.0.0.1:8080`, mientras que el README indica el puerto 3306 de XAMPP. Si no se define `DB_HOST`, la conexión puede fallar. Se recomienda definir siempre `DB_HOST` y `DB_PORT` explícitamente (hallazgo H-01).

4.2 Ejemplo de configuración en Apache

Ejemplo ilustrativo para la sección de configuración de Apache. Los valores reales deben corresponder al entorno.

```
SetEnv DB_HOST 127.0.0.1
SetEnv DB_PORT 3306
SetEnv DB_NAME catalogo_ecommerce
SetEnv DB_USER tienda_app
SetEnv DB_PASS cambiar_por_una_clave_segura
SetEnv MAIL_ENABLED false
```

Se recomienda crear un usuario de base de datos dedicado, con privilegios solo sobre `catalogo_ecommerce`, en lugar de usar `root` en producción.

4.3 Control de versiones

El archivo `.gitignore` excluye `config.php` y todos los `*.sql`, pero luego los vuelve a incluir con `!config.php` y `!bdcatalogo.sql`. Como `config.php` no contiene secretos, puede versionarse. Las credenciales nunca deben escribirse en él.

4.4 Configuración de la tienda

El nombre, los colores, el logo, el banner y los datos de contacto se administran desde el panel (RF-46). Cada guardado inserta una fila nueva en `personalizacion_tienda` y la tienda muestra la fila con mayor `id_personalizacion` (RN-13).

5. Instalación

El procedimiento sigue el README del repositorio. Cada paso incluye una comprobación para confirmar que se completó antes de continuar.

1. **Obtener el código.** Clone el repositorio o descomprima el paquete en el directorio web, por ejemplo `htdocs/TechnoStore` en XAMPP. *Comprobación:* existen `api.php`, `index.php`, `config.php` y `bdcatalogo.sql`.
2. **Instalar dependencias.** Desde la raíz del proyecto ejecute `composer install`. *Comprobación:* existen `vendor/` y `vendor/autoload.php`.
3. **Iniciar los servicios.** Inicie Apache y MySQL en XAMPP. *Comprobación:* `mysql --version` responde y el panel de XAMPP muestra ambos servicios en ejecución.
4. **Importar el esquema.** Importe `bdcatalogo.sql` desde phpMyAdmin (`http://localhost/phpmyadmin`) o desde la línea de comandos. El script crea la base `catalogo_ecommerce`, las tablas, vistas, procedimientos y datos de ejemplo. Antes de importar, elimine la definición duplicada de `sp_buscar_productos` (observación O-08). *Comprobación:* la base contiene las tablas `usuario`, `categoria`, `producto`, `pedido`, `detalle_pedido` y `personalizacion_tienda`.
5. **Definir las variables de entorno** según la sección 4. *Comprobación:* reinicie Apache después de cambiarlas.
6. **Preparar las carpetas de imágenes.** Verifique que `img/` (si se usan imágenes locales) y `img/uploads/` existan o puedan crearse, con permisos de escritura para el usuario del servidor web. *Comprobación:* la subida de una imagen válida desde el panel devuelve una ruta `img/uploads/...`.
7. **Abrir la tienda.** Visite `http://localhost/TechnoStore/`. La primera petición ejecuta la migración de la sección 6. *Comprobación:* el catálogo carga sin mensajes de error.
8. **Retirar las credenciales de ejemplo.** El script SQL crea cuentas con la contraseña provisional `123456` (RN-15, O-01). Cambie esas contraseñas antes de exponer el sistema, o elimine las cuentas en producción.

5.1 Comprobación desde la línea de comandos

```
php -l api.php
mysql -u tienda_app -p -e "USE catalogo_ecommerce; SHOW TABLES;"
curl "http://localhost/TechnoStore/api.php?action=obtener_tienda"
```

La primera orden valida la sintaxis de `api.php`. La segunda lista las tablas de la base. La tercera consulta una acción pública y debe responder con `success: true`. Ajuste el usuario y la ruta a su entorno.

6. Migración de esquema

Al abrirse por primera vez en una sesión de PHP, `api.php` ejecuta la función `migrar()` (línea 80). Su bandera `$_SESSION['mig_ok']` evita repetirla en la misma sesión. La función es idempotente: consulta `information_schema` antes de cada cambio y tolera el error 1060 (columna duplicada).

La migración realiza estos cambios:

- Agrega a `usuario` las columnas `pais`, `ciudad` y `barrio`.
- Agrega a `pedido` las columnas `pais_envio`, `ciudad_envio` y `barrio_envio`.
- Agrega a `detalle_pedido` la columna `precio_unitario` y fija el precio actual a los pedidos confirmados que no lo tenían.
- Agrega a `usuario` la columna `saldo_digital` con valor por defecto 10.000.000,00 y la restricción CHECK ≥ 0.
- Crea las tablas `transaccion_digital` y `comprobante` si no existen.

Es importante saber que `transaccion_digital` y `comprobante` no se crean en `bdcatalogo.sql`, sino en `migrar()`. Si la migración falla, la API responde 500 con el mensaje «No se pudo actualizar la base de datos». Esto suele indicar que el usuario de la base de datos no tiene privilegios de `ALTER` o `CREATE`.

7. Contrato de la API

7.1 Convenciones

- Cada petición indica la operación con el parámetro `action`, por ejemplo `api.php?action=listar_catalogo`.
- Toda respuesta es JSON con la forma `{success: boolean, ...}`. Un error devuelve `{success: false, error: "mensaje"}`.
- Las entradas de texto se recortan con `trim()` antes de validarse.
- El servidor nunca toma el identificador de usuario de la entrada. Siempre usa el de la sesión (RF-13).
- Los errores de base de datos no revelan detalles. La respuesta es «Error de base de datos» con código 500 (RNF-06).

7.2 Códigos HTTP

Código	Significado	Ejemplo de mensaje
200	Operación exitosa	(sin error)
400	Validación fallida o regla de negocio incumplida	«La cantidad debe ser mayor a 0»
401	No hay sesión iniciada	«Debes iniciar sesión»
403	Rol insuficiente o recurso ajeno	«Solo para administradores»
404	Recurso no encontrado o no disponible	«Este producto ya no está disponible»

Código	Significado	Ejemplo de mensaje
500	Error interno de base de datos	«Error de base de datos»

7.3 Ejemplos de petición y respuesta

La confirmación de un pedido con dinero digital requiere sesión activa. Los parámetros se envían en el cuerpo de la petición. El formato exacto del cuerpo (formulario o JSON) debe confirmarse en la función de lectura de entradas de `api.php`; el ejemplo siguiente lo muestra en JSON por claridad.

Petición (ejemplo):

```
{
  "action": "confirmar_pedido",
  "direccion_envio": "Calle 10 # 5-20",
  "pais_envio": "Colombia",
  "ciudad_envio": "Neiva",
  "barrio_envio": "Centro",
  "metodo_pago": "Dinero digital"
}
```

Respuesta exitosa (valores ilustrativos):

```
{
  "success": true,
  "id_pedido": 42,
  "comprobante": "COMP-000017",
  "saldo": 8950000.00
}
```

Respuesta por stock insuficiente (código 400):

```
{
  "success": false,
  "error": "Stock insuficiente de Audífonos Sony (quedan 2)"
}
```

Los métodos Tarjeta crédito, PSE, Nequi y Contraentrega devuelven solo `id_pedido`, y el pedido queda en estado PENDIENTE (RF-19).

7.4 Inventario de acciones

`api.php` define 48 acciones. El SRS documenta 45 de ellas; las acciones `obtener_paises`, `obtener_ciudades` y `obtener_producto` existen en el código y no figuran en el SRS (hallazgo H-09). La tabla agrupa las acciones por módulo.

Módulo	Acciones	Rol	Requerimientos
Catálogo	`listar_catalogo`, `obtener_vista_catalogo`, `buscar_productos`, `listar_categorias`, `detalle_producto`, `obtener_producto`, `obtener_destacados`, `obtener_tienda`	Público	RF-01 a RF-06

Módulo	Acciones	Rol	Requerimientos
Ubicación	`obtener_paises`, `obtener_ciudades`	Público	RNF-09, O-13
Sesión	`registrar`, `login`, `logout`, `sesion`	Público o sesión	RF-07 a RF-10
Perfil	`perfil_obtener`, `perfil_actualizar`, `cambiar_password`	Cliente	RF-11, RF-12
Carrito	`agregar_carrito`, `ver_carrito`, `actualizar_carrito`, `vaciar_carrito`	Cliente	RF-14 a RF-17
Pedidos	`confirmar_pedido`, `cancelar_pedido`, `pedidos_usuario`, `detalle_pedido`	Cliente (detalle: cliente o admin)	RF-18, RF-20 a RF-22
Dinero digital	`saldo_digital`, `recargar_saldo`, `movimientos_saldo`, `obtener_comprobante`	Cliente o admin	RF-23, RF-24, RF-28 a RF-30
Catálogo admin	`admin_productos`, `crear_producto`, `actualizar_producto`, `actualizar_stock`, `eliminar_producto`, `crear_categoria`, `actualizar_categoria`, `eliminar_categoria`, `subir_imagen`, `restaurar`, `listar_eliminados`	Administrador	RF-31 a RF-37, RF-38
Pedidos admin	`admin_pedidos`, `cambiar_estado_pedido`	Administrador	RF-39 a RF-41
Usuarios	`listar_usuarios`, `cambiar_rol`, `admin_actualizar_usuario`, `eliminar_usuario`	Administrador	RF-42 a RF-45
Tienda	`tienda_admin`, `guardar_tienda`	Administrador	RF-46
Reportes	`reporte.php?tipo=...` (archivo aparte, no `api.php`)	Administrador	RF-47 a RF-49

La acción `restaurar` recibe el parámetro `tipo` con los valores `producto`, `categoria` o `usuario`. Los valores `categoria`, `producto` y `usuario` que aparecen en el código son sub-casos de esa acción, no acciones independientes.

8. Modelo de datos

La base de datos se llama `catalogo_ecommerce`, usa el motor InnoDB y la collation `utf8mb4_unicode_ci`. Las claves foráneas usan RESTRICT, salvo las de `detalle_pedido` y `personalizacion_tienda`, que usan CASCADE (RNF-01).

8.1 Tablas

Tabla	Origen	Clave primaria	Restricciones relevantes
usuario	bdcatalogo.sql y migrar()	id_usuario	correo UNIQUE. rol ENUM(ADMINISTRADOR, CLIENTE), por defecto CLIENTE. saldo_digital DECIMAL(12,2), por defecto 10000000.00, con CHECK ≥ 0. activo
categoria	bdcatalogo.sql	id_categoria	nombre UNIQUE. activa por defecto TRUE
producto	bdcatalogo.sql	id_producto	FK a categoria (RESTRICT). CHECK precio ≥ 0 y stock ≥ 0. activo. Índices en id_categoria y nombre
pedido	bdcatalogo.sql	id_pedido	FK a usuario (RESTRICT). estado ENUM(CARRITO, PENDIENTE, PAGADO, ENVIADO, ENTREGADO, CANCELADO)
detalle_pedido	bdcatalogo.sql	id_detalle	FK a pedido (CASCADE) y producto (RESTRICT). UNIQUE (id_pedido, id_producto). CHECK cantidad > 0. precio_unitario NULL mientras el pedido es carrito
personalizacion_tienda	bdcatalogo.sql	id_personalizacion	FK a usuario (CASCADE). Colores VARCHAR(7), por defecto #1F4E79 y #FFF2CC
transaccion_digital	migrar()	id_transaccion	tipo ENUM(RECARGA, PAGO, REEMBOLSO). CHECK monto > 0. id_pedido NULL en recargas. Registra saldo_anterior y saldo_posterior
comprobante	migrar()	id_comprobante	numero VARCHAR(20) UNIQUE, con formato

Tabla	Origen	Clave primaria	Restricciones relevantes
			COMP-000000. `id_pedido` UNIQUE. FK a `pedido`, `usuario` y `transaccion_digital` (RESTRICT)

Las columnas `pais`, `ciudad` y `barrio` de `usuario`, las columnas `pais_envio`, `ciudad_envio` y `barrio_envio` de `pedido` y la columna `precio_unitario` de `detalle_pedido` también se agregan en `migrar()`.

Todos los importes son DECIMAL(12,2). En PHP se redondean con `round($valor, 2)` antes de operar. Los montos nunca se guardan con tipos de coma flotante (RNF-03).

8.2 Relaciones

- Un `usuario` tiene cero o más `pedido`. Como máximo un pedido por usuario está en estado CARRITO (RN-12).
- Un `pedido` tiene una o más líneas en `detalle_pedido`. Un `producto` puede aparecer en muchas líneas.
- Un `pedido` pagado con dinero digital genera exactamente un `comprobante` (RN-04).
- Cada movimiento de saldo queda en `transaccion_digital`. La aplicación no modifica ni elimina esos registros (RNF-10).

8.3 Vistas

- **`v_catalogo_publico`:** productos activos, de categorías activas y con stock mayor a cero. Base de RF-01 y RF-04.
- **`v_productos_destacados`:** la vista anterior filtrada por `destacado = TRUE`. Base de RF-05. Está definida dos veces en `bdcatalogo.sql` (líneas 242 y 635); el resultado es el mismo, pero conviene eliminar una definición (hallazgo H-05).
- **`v_stock_bajo`:** productos activos con stock menor o igual a 5. No la usa ni `api.php` ni `index.php` (O-10).
- **`v_detalle_pedido_completo`:** líneas de pedido con subtotal calculado.

8.4 Procedimientos almacenados

`bdcatalogo.sql` define 21 procedimientos distintos. El procedimiento `sp_buscar_productos` aparece dos veces (O-08).

Los procedimientos de escritura validan sus entradas con `SIGNAL SQLSTATE '45000'` y un mensaje en español. La API convierte esa señal en una respuesta 400 (RNF-06).

- **Escritura:** `sp_registrar_usuario`, `sp_crear_categoria`, `sp_crear_producto`, `sp_agregar_al_carrito`, `sp_actualizar_stock`, `sp_actualizar_categoria`, `sp_eliminar_categoria`, `sp_actualizar_producto`, `sp_eliminar_producto`, `sp_cambiar_rol`, `sp_eliminar_usuario`.

- **Lectura:** `sp_login_usuario`, `sp_listar_catalogo`, `sp_buscar_productos`, `sp_productos_por_categoria`, `sp_pedidos_usuario`, `sp_obtener_tienda`, `sp_listar_categorias`, `sp_admin_listar_productos`, `sp_obtener_producto`, `sp_listar_usuarios`.

9. Lógica transaccional

Las operaciones que modifican stock o saldo se ejecutan dentro de una transacción. Cada transacción bloquea las filas que modifica con `SELECT ... FOR UPDATE`, de modo que dos peticiones simultáneas no leen el mismo stock o saldo a la vez. Si cualquier paso falla, la transacción completa se revierte (RN-03). Las funciones de esta sección están en `api.php`.

9.1 Confirmación de pedido (`confirmar_pedido`, RF-18)

1. Iniciar la transacción.
2. Bloquear el pedido. Si su estado no es CARRITO, responder «Este pedido ya fue procesado». Así, una segunda petición del mismo carrito se rechaza.
3. Bloquear las líneas del detalle y sus productos. Si el carrito está vacío, rechazar con «Tu carrito está vacío».
4. Verificar que cada cantidad sea menor o igual al stock. Si no, rechazar con «Stock insuficiente de {producto} (quedan {stock})».
5. Descontar el stock de cada producto.
6. Copiar `producto.precio` en `detalle_pedido.precio_unitario` (RN-02).
7. Actualizar el pedido. El estado es PAGADO si el método es Dinero digital, o PENDIENTE en los demás casos. Registrar `fecha_compra`, `metodo_pago` y la dirección.
8. Si el método es Dinero digital, ejecutar `moverSaldo` (tipo PAGO) y `emitirComprobante` dentro de la misma transacción.
9. Confirmar la transacción (COMMIT). Después se envía el correo de RF-50, fuera de la transacción. Un fallo de correo no revierte el pedido (RNF-09).

Cualquier excepción en los pasos 2 a 8 ejecuta ROLLBACK completo. El pedido vuelve a CARRITO y el stock no cambia.

El siguiente esquema es ilustrativo. Resume el orden de las operaciones y no reproduce el código fuente:

```
START TRANSACTION;
SELECT estado FROM pedido WHERE id_pedido = ? FOR UPDATE;
SELECT id_producto, cantidad FROM detalle_pedido WHERE id_pedido = ? FOR UPDATE;
-- verificar stock, descontar y congelar precios
UPDATE pedido SET estado = ?, metodo_pago = ?, fecha_compra = NOW() WHERE id_pedido =
?;
-- si el método es Dinero digital: moverSaldo (PAGO) y emitirComprobante
COMMIT;
```

9.2 Función `moverSaldo` (línea 187)

`moverSaldo(usuario, tipo, monto, id_pedido)` es la única vía para modificar `saldo_digital`. Debe ejecutarse dentro de una transacción. Bloquea la fila del usuario con `FOR UPDATE`, calcula el nuevo

saldo y registra en `transaccion_digital` el saldo anterior y el posterior. Si el saldo resultante sería negativo, lanza `RuntimeException` con el mensaje «Saldo digital insuficiente: tienes $X y el pedido cuesta $Y» (línea 198). Esa excepción revierte toda la compra, así que no se descuenta stock ni cambia el estado (RN-09).

9.3 Otras funciones

- **`totalPedido()` (línea 209):** suma `cantidad × precio_unitario` de las líneas del pedido, que usan los precios congelados.
- **`emitirComprobante()` (línea 215):** inserta el comprobante y después asigna `numero` con el formato `COMP-` y seis dígitos. Como el número depende del id autoincremental, se asigna en un segundo paso. La restricción UNIQUE sobre `id_pedido` impide un segundo comprobante por pedido (RN-04).
- **`reembolsarPedido()` (línea 227):** es idempotente. No hace nada si no existe un movimiento PAGO para el pedido o si ya existe un REEMBOLSO. En otro caso ejecuta `moverSaldo` con tipo REEMBOLSO (RN-05).

9.4 Cancelación por el cliente (`cancelar_pedido`, línea 476)

El cliente solo puede cancelar pedidos en estado PENDIENTE. Si el pedido está en otro estado, la respuesta es «Solo puedes cancelar pedidos pendientes» (RN-07). Al cancelar, el stock vuelve al inventario y el estado pasa a CANCELADO. Después se envía el correo de RF-50.

9.5 Máquina de estados de un pedido (RN-06)

La matriz de transiciones está en el arreglo `$flujo` de `cambiar_estado_pedido` (línea 558).

Estado actual	Transiciones permitidas	Quién
CARRITO	PENDIENTE o PAGADO	Sistema, al confirmar (RF-18)
PENDIENTE	PAGADO, CANCELADO	Administrador. El cliente solo cancela (RF-20)
PAGADO	ENVIADO, CANCELADO	Administrador
ENVIADO	ENTREGADO, CANCELADO	Administrador
ENTREGADO	Ninguna (estado terminal)	—
CANCELADO	Ninguna (estado terminal)	—

Cualquier transición fuera de la matriz se rechaza con «No se puede pasar un pedido de {actual} a {nuevo}». Al cancelar desde el administrador (RF-41), el stock vuelve al inventario y, si el pedido se pagó con dinero digital, se ejecuta el reembolso.

10. Seguridad

10.1 Controles implementados

Amenaza	Control	Evidencia en el código	Referencia
Robo de contraseñas	`password_hash(PASSWORD_DEFAULT)` (bcrypt) y `password_verify`	`api.php`, registro y login	RNF-04
Fijación de sesión	`session_regenerate_id(true)` tras registro, inicio de sesión y cambio de contraseña	`api.php:320`, `:343`, `:650`	RNF-05
Inyección SQL	Sentencias preparadas de PDO con `ATTR_EMULATE_PREPARES` en false	`config.php`	RNF-06
Fuga de detalles internos	Errores de base de datos genéricos (500)	`q()` y `fail()` en `api.php`	RNF-06
Credenciales en el código	Lectura desde variables de entorno	`config.php`	O-07
Acceso no autorizado	`needLogin()` y `needAdmin()`. Un cliente solo consulta pedidos y comprobantes propios	`api.php:252` y `:258`	RF-13
Escalamiento de privilegios	Ningún endpoint toma el id de usuario de la entrada. Un administrador no puede cambiar su propio rol ni eliminar su cuenta	`api.php`, `cambiar_rol` y `eliminar_usuario`	RN-11
Inyección HTML en correos	`htmlspecialchars` con `ENT_QUOTES`	`mailer.php`	RNF-07
Carga de archivos maliciosos	Límite de 2 MB, validación con `getimagesize()`, nombre aleatorio de 16 caracteres y extensión asignada por el servidor	`api.php:694` a `:701`	RF-36
URLs peligrosas en productos	La validación rechaza comillas, espacios, paréntesis y signos menor y mayor que	`api.php:240` (`urlSegura`)	RNF-07

10.2 Estado de las observaciones

La tabla contrasta cada observación del SRS con el código del proyecto. El estado es el verificado en esta versión del manual.

ID	Severidad	Estado verificado	Evidencia y acción recomendada

ID	Severidad	Estado verificado	Evidencia y acción recomendada
O-01	Alta	**Abierta**	`api.php:315` acepta «123456» cuando el hash empieza por `$2y$10$hashEjemplo`. Lo mismo ocurre en `cambiar_password` (línea 646). Eliminar esas ramas y retirar las cuentas de ejemplo.
O-07	Alta	**Mitigada en este código**	`config.php` ya no contiene credenciales. Antes de dar por cerrada la observación, revise el historial de Git: si alguna credencial se subió en un commit anterior, debe rotarse.
O-02	Media	**Abierta**	No existe token anti-CSRF en el código y no se configura `session.cookie_samesite`. Añadir un token por sesión validado en cada POST.
O-03	Media	**Abierta**	No hay límite de intentos de inicio de sesión. Limitar intentos por ventana de tiempo y por IP, y registrar los fallos.
O-05	Media	**Abierta**	`cancelar_pedido` (línea 482) solo permite PENDIENTE. Los Términos dicen «mientras no haya sido despachado». No existe el flujo de retracto de 5 días de la Ley 1480 de 2011.
O-06	Media	**Abierta**	`actualizar_stock` (línea 524) llama a `sp_actualizar_stock` con el nombre del producto, que no es único. Usar `id_producto`.
O-04	Media	**Abierta**	Tarjeta, PSE, Nequi y Contraentrega solo registran el pedido. Integrar un proveedor de pagos antes de uso productivo.

Las observaciones O-01, O-02 y O-03 deben corregirse antes de un despliegue público. O-07 requiere revisar el historial de Git y, si fuera necesario, rotar credenciales.

11. Reportes y notificaciones

11.1 Reportes en Excel (`reporte.php`)

Los reportes se generan en `reporte.php`, no en `api.php`. Requieren sesión de administrador: sin sesión responde 401 y con otro rol responde 403. Si la generación es correcta, devuelven un .xlsx con la cabecera `Content-Disposition`. Si hay error, devuelven JSON con `success: false`. La zona horaria es America/Bogotá.

Reporte	Valor de `tipo`	Contenido	Requerimiento
Pedidos por	`pedidos_entregados`, `pedidos_pendientes`,	Hoja Pedidos y hoja Detalle de productos,	RF-47

Reporte	Valor de `tipo`	Contenido	Requerimiento
estado	`pedidos_enviados`	ambas con fila de totales	
Usuarios con pedidos	`usuarios_con_pedidos`	Pedidos realizados, cancelados, total comprado sin cancelados y último pedido	RF-48
Usuarios sin pedidos	`usuarios_sin_pedidos`	Usuarios sin ningún pedido distinto de CARRITO	RF-48
Ranking de productos	`productos_mas_menos_comprados`	Unidades, pedidos, ingresos, stock y estado. Considera PENDIENTE, PAGADO, ENVIADO y ENTREGADO	RF-49

Los importes usan `COALESCE(d.precio_unitario, pr.precio)`, de modo que un reporte histórico no cambia cuando cambia el precio actual de un producto. El libro se genera con la clase `XlsxWriter` de `xlsx.php`, que escribe el formato OOXML directamente y usa la extensión ZIP de PHP.

11.2 Notificaciones por correo (`mailer.php`, RF-50)

La función `notificarPedido` se invoca después del COMMIT en `confirmar_pedido`, `cancelar_pedido` y `cambiar_estado_pedido`. Envía el correo solo si `MAIL_ENABLED` es `true`, si el estado está en la lista `MAIL_ESTADOS` y si el correo del cliente es válido. Captura cualquier excepción y devuelve `false`. Como el envío ocurre después de la transacción, un fallo de SMTP nunca deshace el cambio de estado (RNF-09).

12. Plan de pruebas

Los casos verifican las reglas más sensibles del sistema. Deben ejecutarse en un entorno de prueba con datos de muestra, nunca con datos reales. Los mensajes esperados son los del código.

ID	Caso	Procedimiento	Resultado esperado
PR-01	Confirmación concurrente	Enviar dos `confirmar_pedido` en paralelo con el mismo carrito	Solo una respuesta es exitosa. La otra responde 400 «Este pedido ya fue procesado»
PR-02	Último stock	Dos pedidos distintos por la última unidad, en paralelo	Un pedido se confirma. El otro responde 400 «Stock insuficiente». El stock nunca es negativo
PR-03	Saldo insuficiente	Pagar con dinero digital un total mayor al saldo	400 con el mensaje de saldo. El stock no cambia, el pedido sigue en CARRITO y no hay comprobante
PR-04	Reembolso idempotente	Cancelar dos veces un pedido pagado con saldo	El saldo se devuelve una sola vez. Existe un único movimiento REEMBOLSO

ID	Caso	Procedimiento	Resultado esperado
PR-05	Acceso sin sesión	Llamar `agregar_carrito` sin sesión	401 «Debes iniciar sesión»
PR-06	Pedido ajeno	Un cliente consulta `detalle_pedido` de otro cliente	403 «Ese pedido no es tuyo»
PR-07	Cancelación fuera de regla	Un cliente cancela un pedido PAGADO	400 «Solo puedes cancelar pedidos pendientes»
PR-08	Autoprotección de administrador	Un administrador cambia su propio rol	400 «No puedes cambiar tu propio rol»
PR-09	Validación de imagen	Subir un archivo de texto y, por separado, una imagen de más de 2 MB	400 «Formato no válido» y 400 «La imagen supera el máximo de 2 MB»
PR-10	Rendimiento del catálogo	50 peticiones concurrentes a `listar_catalogo` con 1.000 productos	Percentil 95 menor a 500 ms. Es un objetivo de diseño, no una medición (RNF-08)
PR-11	Migración	Abrir la tienda con una base sin `transaccion_digital`	Las tablas y columnas se crean. Una segunda sesión no repite la migración

Registre en cada prueba la fecha, la versión del código, el resultado obtenido y las evidencias (respuestas JSON y consultas a la base de datos).

13. Solución de problemas

Síntoma	Causa probable	Acción
La conexión falla al abrir la tienda	`DB_HOST` apunta al puerto 8080 por defecto, o MySQL no está iniciado	Definir `DB_HOST=127.0.0.1` y `DB_PORT=3306`. Revisar el log de errores de PHP, que registra «Error de conexión a MySQL»
La API responde 500 «Error de base de datos»	Credenciales incorrectas, base no importada o usuario sin privilegios	Verificar `DB_USER`, `DB_PASS` y la existencia de `catalogo_ecommerce`
La API responde 500 «No se pudo actualizar la base de datos»	La migración no tiene privilegios de ALTER o CREATE	Otorgar al usuario de base de datos los privilegios necesarios sobre `catalogo_ecommerce`
Todas las acciones responden 401 «Debes iniciar	La cookie de sesión no se envía o la sesión expiró	Revisar que el navegador acepte cookies y que PHP pueda escribir en

Síntoma	Causa probable	Acción
sesión»		`session.save_path`
El catálogo aparece vacío	No hay productos con stock mayor a cero o las categorías están inactivas	Consultar `v_catalogo_publico` y la acción `admin_productos`
Un producto no se puede restaurar	Su categoría sigue dada de baja	Restaurar primero la categoría con `restaurar` y `tipo=categoria`
La subida de imágenes falla	`img/uploads` no existe o no tiene permisos de escritura	Crear la carpeta y asignar permisos al usuario del servidor web
No llegan correos	`MAIL_ENABLED` es `false`, o las credenciales SMTP son incorrectas	Verificar las variables `MAIL_*`. El pedido queda guardado de todos modos (RNF-09)
Los países o ciudades no cargan	restcountries.com o Nominatim no responden	Es el comportamiento esperado tras 12 segundos de espera. El usuario puede escribir país y ciudad a mano

14. Operación y mantenimiento

- **Baja lógica.** Productos, categorías y usuarios se desactivan con `activo` o `activa` en FALSE. La aplicación no ejecuta DELETE sobre estas tablas. Para recuperar un elemento se usan `restaurar` y `listar_eliminados` (RN-14).
- **Respaldos.** Haga respaldos periódicos de `catalogo_ecommerce`, con prioridad en `transaccion_digital` y `comprobante`, que son registros de trazabilidad financiera.
- **Imágenes.** La carpeta `img/uploads` crece con cada carga. Defina una política de limpieza de archivos que ya no estén referenciados por ningún producto.
- **Registros de la aplicación.** El código no define un sistema de logs propio; usa `error_log()` de PHP. Se recomienda registrar también los errores de transacción y los fallos de correo.
- **Cambios de esquema.** Para agregar una columna o tabla, extienda `migrar()` con una consulta a `information_schema` antes del `ALTER TABLE`, como lo hace el código actual (RNF-11).
- **Archivos residuales.** El archivo `configof.php` no tiene contenido técnico y no es usado por el sistema. Debe eliminarse antes de la entrega (hallazgo H-04).

15. Hallazgos de la verificación contra el código

La verificación comparó el SRS v1.0 con los archivos del proyecto (`Nueva_carpeta.zip`). Los hallazgos siguientes son diferencias o problemas que el SRS no registra o registra de forma distinta.

ID	Hallazgo	Evidencia	Severidad	Acción recomendada
H-01	El valor por defecto de `DB_HOST` es `127.0.0.1:8080`, pero el README	`config.php`;	Media	Cambiar el valor por defecto a

ID	Hallazgo	Evidencia	Severidad	Acción recomendada
	indica el puerto 3306 de XAMPP.	README, paso 3		`127.0.0.1` con `DB_PORT=3306`, o documentar el puerto 8080
H-02	La versión mínima de PHP no coincide: el README indica 7.4 o superior y el SRS (RNF-12) indica 8.0 o superior.	README, «Requisitos del Sistema»; SRS RNF-12	Media	Definir una sola versión y probarla. Este manual no ejecutó el código en PHP 7.4
H-03	El archivo `configof.php` contiene un texto sin relación con el proyecto y no es referenciado por ningún archivo.	`configof.php`; búsqueda en todo el proyecto	Baja	Eliminar el archivo antes de la entrega
H-04	El procedimiento `v_productos_destacados` está definido dos veces en `bdcatalogo.sql`.	`bdcatalogo.sql` líneas 242 y 635	Baja	Eliminar una de las dos definiciones
H-05	Las acciones `obtener_paises`, `obtener_ciudades` y `obtener_producto` existen en `api.php` y no están en el SRS.	`api.php`	Baja	Agregarlas a la matriz de trazabilidad del SRS
H-06	Las tablas `transaccion_digital` y `comprobante` no están en `bdcatalogo.sql`. Solo las crea `migrar()`.	`api.php` líneas 131 y 140	Media	Incorporarlas al script SQL o documentar que la migración es obligatoria
H-07	El repositorio incluye `Anexos/SRS/SRS_TechnoStore.pdf` (33 KB), que es distinto del SRS entregado (619 KB).	`Anexos/SRS/`	Baja	Confirmar cuál es la versión vigente y actualizar el repositorio
H-08	`config.php` ya no contiene credenciales, pero el historial de Git no se revisó.	`config.php`; `.gitignore`	Alta	Revisar el historial. Si hay credenciales expuestas, rotarlas
H-09	`README.md` y el SRS describen la confirmación de pedidos y el resto de funciones de forma coherente	Comparación general	Informativa	Ninguna

ID	Hallazgo	Evidencia	Severidad	Acción recomendada
	con el código.			

Las observaciones del SRS se confirmaron con la tabla de la sección 10.2. La marca inconsistente (O-09) también se confirma: `index.php` contiene «TecnoShop» una vez y «TechnoStore» cinco veces.

15.1 Plan de corrección

1. **Inmediato:** O-01 (contraseña provisional), H-08 (historial de credenciales) y H-01 (`DB_HOST`).
2. **Antes de producción:** O-02 (CSRF), O-03 (límite de intentos), O-06 (identificación por id), O-05 (alineación legal y retracto) y H-02 (versión de PHP).
3. **Antes de uso con clientes reales:** O-04 (integración de pagos reales).
4. **Limpieza:** H-03, H-04, H-06, H-07 y O-08 a O-13.

16. Glosario

- **API:** interfaz de programación del backend (`api.php`). Responde en JSON.
- **Baja lógica:** desactivación de un registro con una bandera, sin borrarlo de la base de datos.
- **CHECK:** restricción de integridad que la base de datos aplica a cada fila.
- **Comprobante:** documento virtual numerado (COMP-000001) que respalda un pago con dinero digital. No tiene validez fiscal.
- **COP:** peso colombiano, moneda única del sistema.
- **Dinero digital:** saldo simulado de cada usuario, con el que se pagan pedidos dentro de la tienda.
- **FOR UPDATE:** cláusula de SQL que bloquea las filas leídas hasta el final de la transacción.
- **Migración:** cambios de esquema que la aplicación aplica automáticamente en la primera sesión (`migrar()`).
- **PDO:** PHP Data Objects, capa de acceso a la base de datos con sentencias preparadas.
- **Procedimiento almacenado:** rutina guardada en la base de datos que encapsula una operación.
- **RF, RN, RNF:** requerimiento funcional, regla de negocio y requerimiento no funcional, según el SRS.
- **Transacción:** conjunto de operaciones que se confirman todas juntas o se revierten todas.

17. Referencias

- SRS de TechnoStore, versión 1.0 (noviembre de 2026).
- Código del proyecto: `api.php`, `index.php`, `config.php`, `mailer.php`, `reporte.php`, `xlsx.php`, `bdcatalogo.sql`, `composer.json` y `README.md`.
- Diagrama entidad-relación: `Anexos/MER/MER OFICIAL.drawio`.
- Parches `dinero-digital.patch` y `frontend-dinero-digital.patch`.
- Ley 1581 de 2012: protección de datos personales (habeas data).
- Ley 1480 de 2011: Estatuto del Consumidor.